

20 Juni, 2026

Sistem Inventarisasi Laboratorium dengan Sistem Database Berbasis Mikrokontroler Arduino UNO

Tugas Besar EL2008 - Kelompok 6 K03

Anggota Kelompok:



Muhammad Ammar Hanif
(13224087)



Hufnagel Ruben Lenard S.
(13224015)



Christian Jonathan Hutajulu
(13224108)



Maria Vanessa Soritan
(13224078)

Daftar Isi

A. Deskripsi & Dekomposisi Masalah

- Optimisasi Memori Arduino
- Pemecahan Sistem

B. Deskripsi Sistem

- DFD
- Fungsi Sistem

C. Arsitektur Memori Arduino

- Memori Volatile
- Memori Non-Volatile

D. Studi Kasus

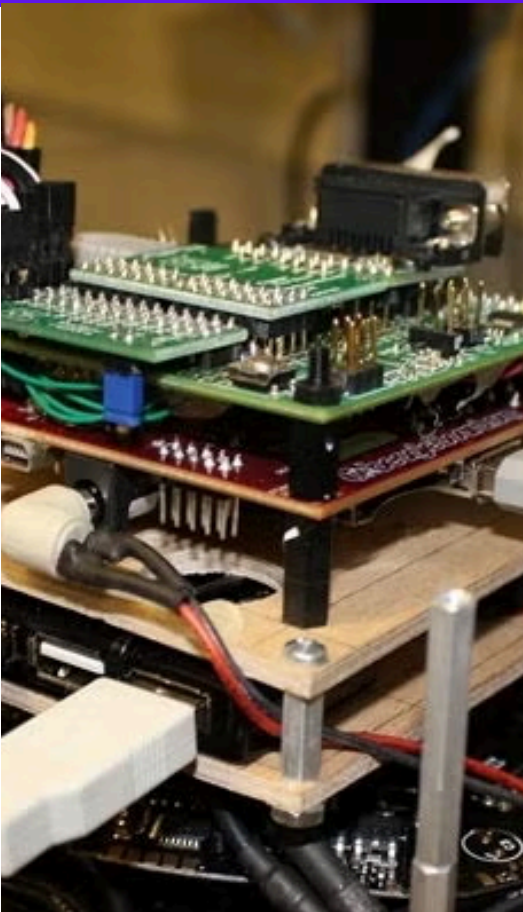
E. Keterbatasan Sistem

F. Kesimpulan & Saran

G. Demonstrasi

H. Pembagian Kerja & Kontribusi

Deskripsi Masalah



Inventarisasi Laboratorium

Laboratorium Embedded Systems memiliki banyak komponen dan peralatan yang digunakan untuk praktikum, penelitian, dan proyek mahasiswa. Saat ini pencatatan inventaris masih dilakukan secara manual sehingga pencarian data, pembaruan stok, dan pemantauan kondisi barang menjadi kurang efisien. Oleh karena itu, dikembangkan sistem inventaris digital berbasis Arduino Uno yang mampu mengelola data barang secara terstruktur dan memaksimalkan penggunaan memori perangkat.

Solusi:

Sistem Inventaris Digital Berbasis Arduino Uno

Pencarian berdasarkan ID Barang

Manajemen stok dan status barang

Database hemat memori dengan Linked List

Akses melalui Serial Monitor



Dekomposisi Masalah

Berikut merupakan semua atribut data yang digunakan dalam sistem:

No	Data	Format Data	Representasi Data (bit)	Keterangan
1	ID	0-127	int 7-bit	Identifier unik tiap barang
2	Nama Item	Maks. 8 karakter (A-Z, a-z, 0-9)	8 × custom 6-bit char (48-bit / 6 byte total)	Nama komponen
3	Kategori	0-3 (0: Microcontroller, 1: Aktuator, 2: Sensor, 3: Lainnya)	int 2-bit	Microcontroller / Aktuator / Sensor / Lainnya
4	Lokasi (Nomor Rak)	0-63	int 6-bit	Posisi rak penyimpanan barang
5	Jumlah Tersedia	0-63	int 6-bit	Stok barang yang tersedia untuk dipinjam
6	Jumlah	0-63	int 6-bit	Jumlah unit barang yang sedang dipinjam
7	Jumlah Rusak	-	Tidak disimpan	Jumlah unit barang dalam kondisi rusak
8	Pemilik Barang	Tepat 3 karakter (A-Z saja)	3 × custom 5-bit char (15-bit, dikemas dalam 1 variabel 16-bit)	(kami asumsikan Laboratorium Embedded Systems merupakan pemilik semua barang)
9	PIC Barang	0-127	int 7-bit	Penanggung jawab barang

Optimasisasi Memori Arduino

Karena data inventaris harus tetap tersimpan meskipun Arduino Uno dimatikan, sistem menggunakan EEPROM internal (1 KB) sebagai media penyimpanan non-volatile. Kapasitas EEPROM yang terbatas mendorong perancangan struktur data yang sangat efisien agar jumlah item yang dapat disimpan menjadi maksimal.

Strategi Optimasi:

- Menghapus atribut yang tidak esensial, seperti data pemilik barang yang diasumsikan selalu milik Laboratorium Embedded Systems.
- Membatasi rentang nilai dan ukuran data (bit-width constraint) sehingga setiap atribut hanya menggunakan jumlah bit minimum yang benar-benar diperlukan.
- Menggunakan ID numerik sebagai key utama untuk mengakses data inventaris secara sederhana dan hemat memori.



Hasil:

- ✔ Data tetap tersimpan setelah power off
- ✔ Penggunaan EEPROM lebih efisien
- ✔ Kapasitas penyimpanan item menjadi lebih maksimal

Pemecahan Sistem

Arduino UNO juga memiliki keterbatasan bukan hanya memori penyimpanan, namun juga pada kemampuan pemrosesan dan tampilan. Maka dari itu, kami memecah program kami menjadi 2 bagian besar sistem, yaitu:

Sistem Database Arduino UNO

database.h & database.c

main.c

Makefile

Fitur Utama:

- Penyimpanan dan pengelolaan data inventaris pada EEPROM
- Komunikasi dua arah melalui Serial Monitor (request & response)
- Validasi data dan penanganan error secara otomatis

Error Handling:

- ID duplikat
- ID tidak ditemukan
- Stok tidak mencukupi
- Memori hampir penuh
- Kapasitas linked list penuh

Sistem Client Interface (terminal)

main.c

app_logic.c & app_logic.h

user_interface.c & user_interface.h

serial_comm.c & serial_comm.h

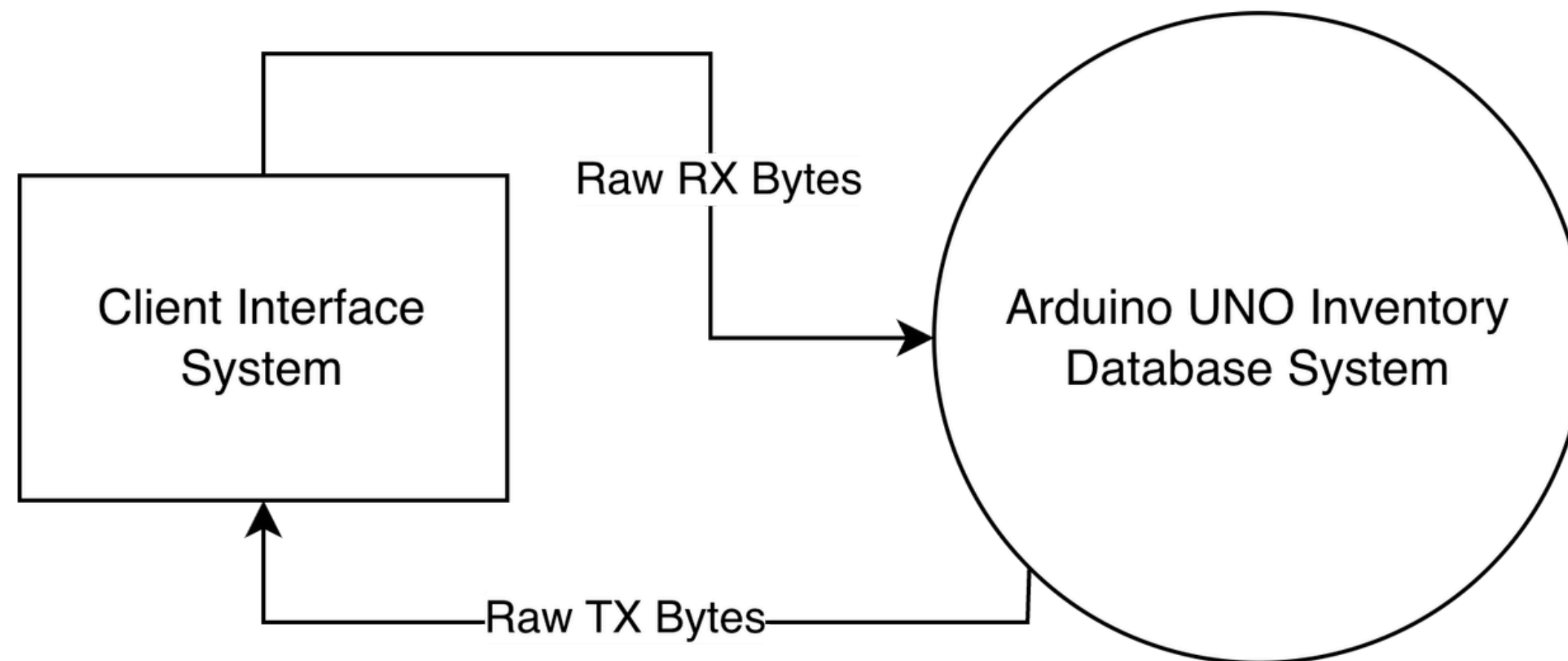
Fitur Utama:

- Menyediakan menu dan interaksi pengguna
- Melakukan validasi input sebelum dikirim ke Arduino
- Mengirim perintah dan menampilkan respons melalui komunikasi serial

Makefile & build.bat

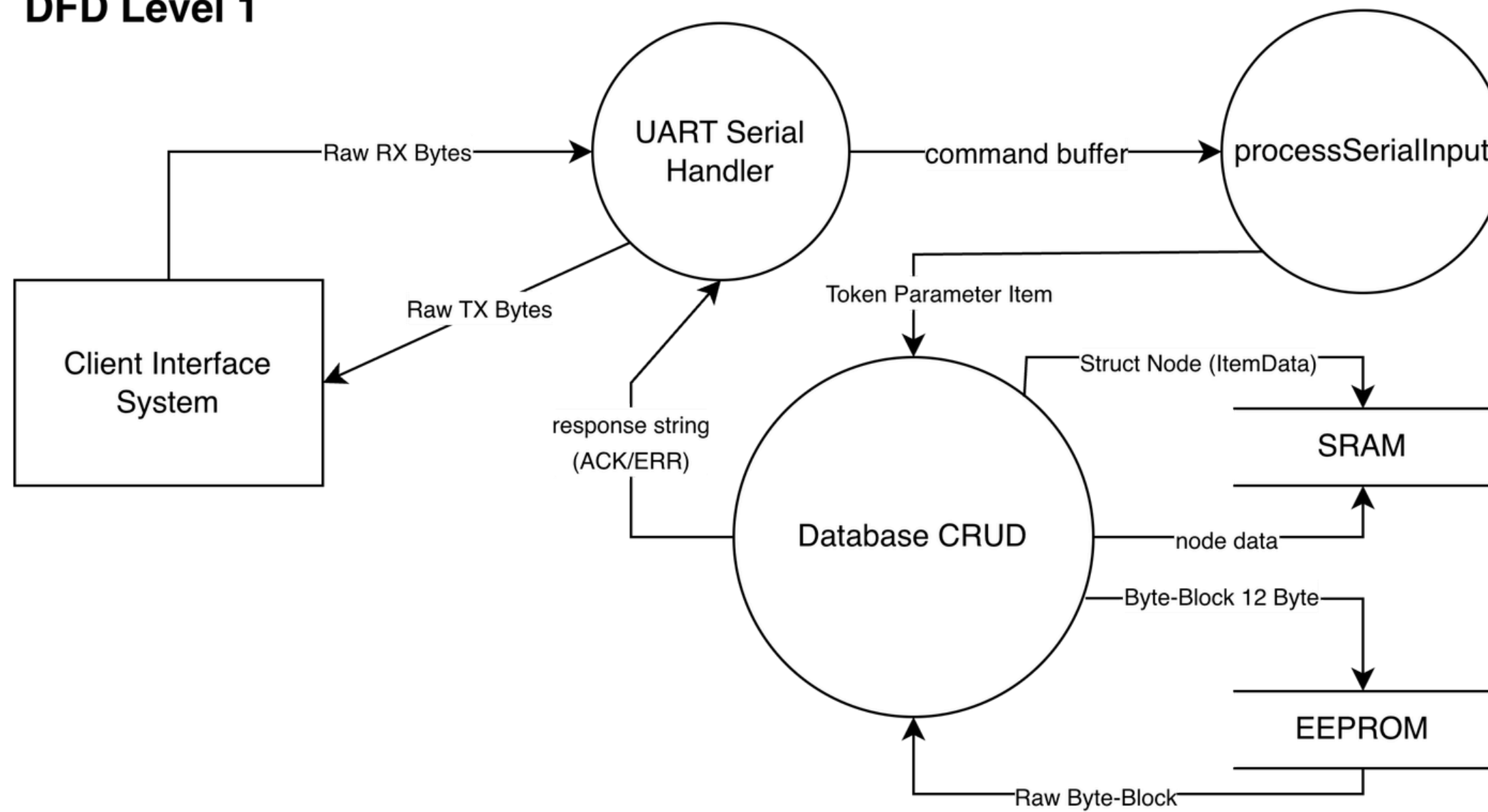
DFD Level 0 Arduino UNO System

DFD Level 0



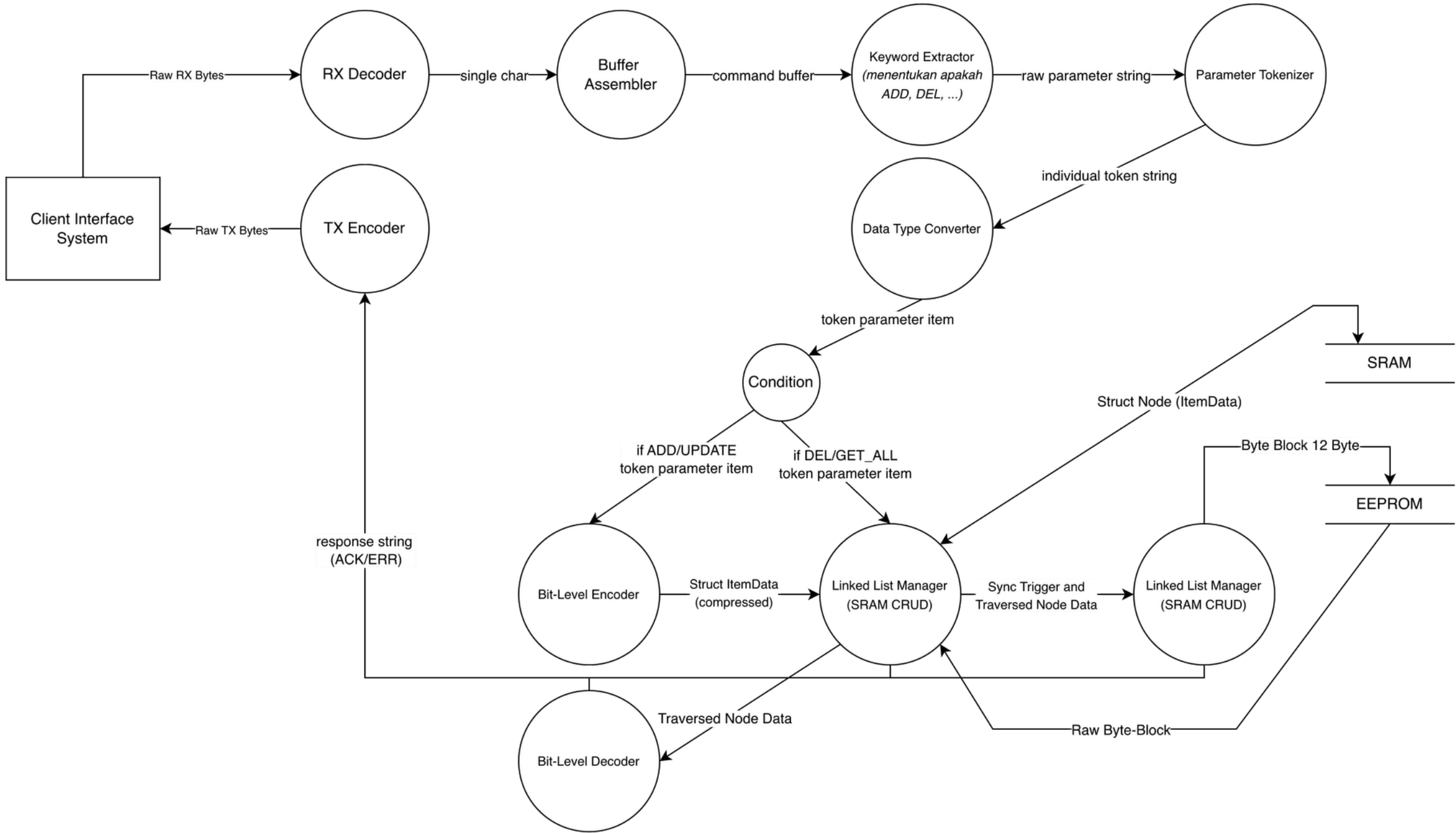
DFD Level 1 Arduino UNO System

DFD Level 1

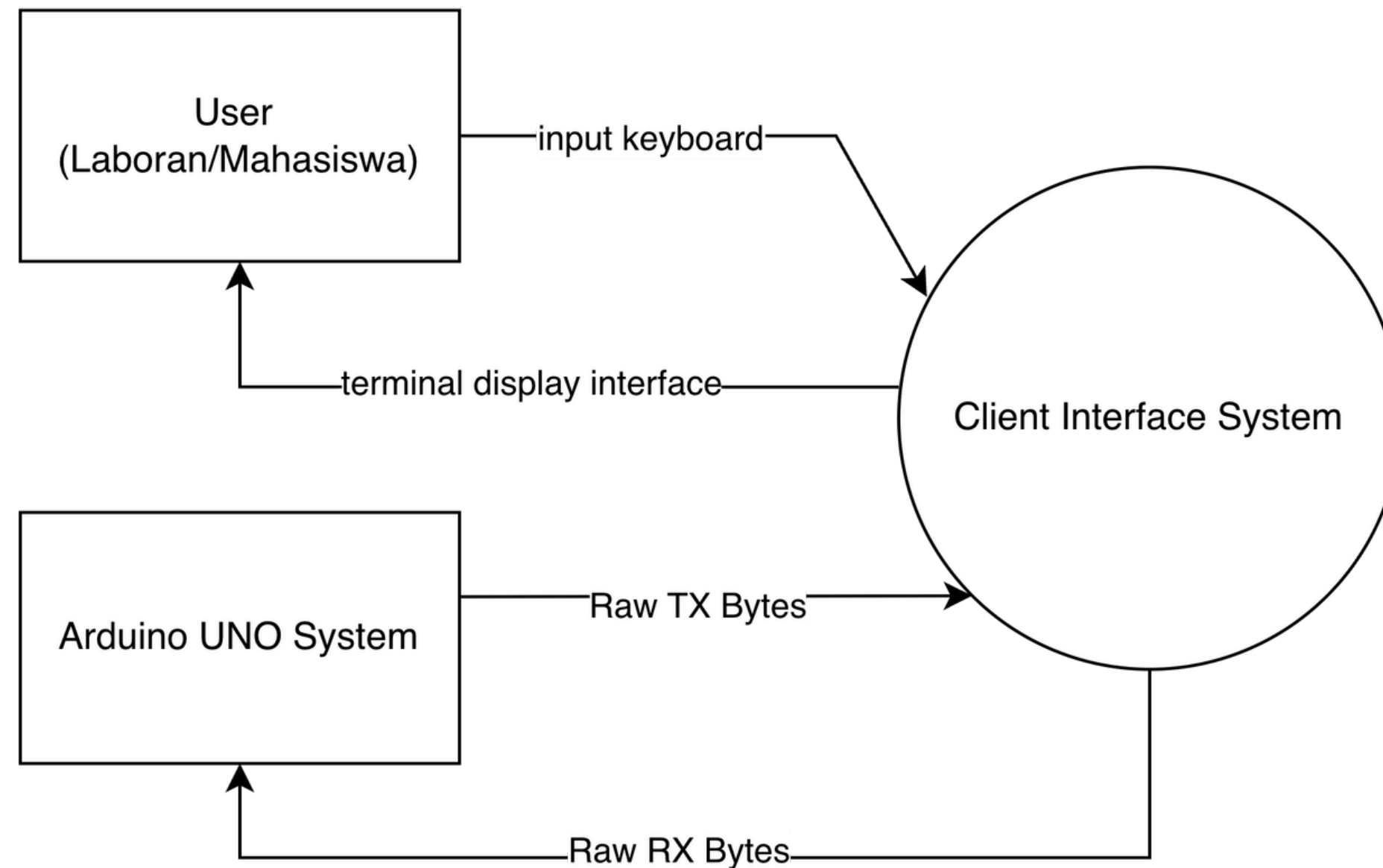


DFD Level 2 Arduino UNO System

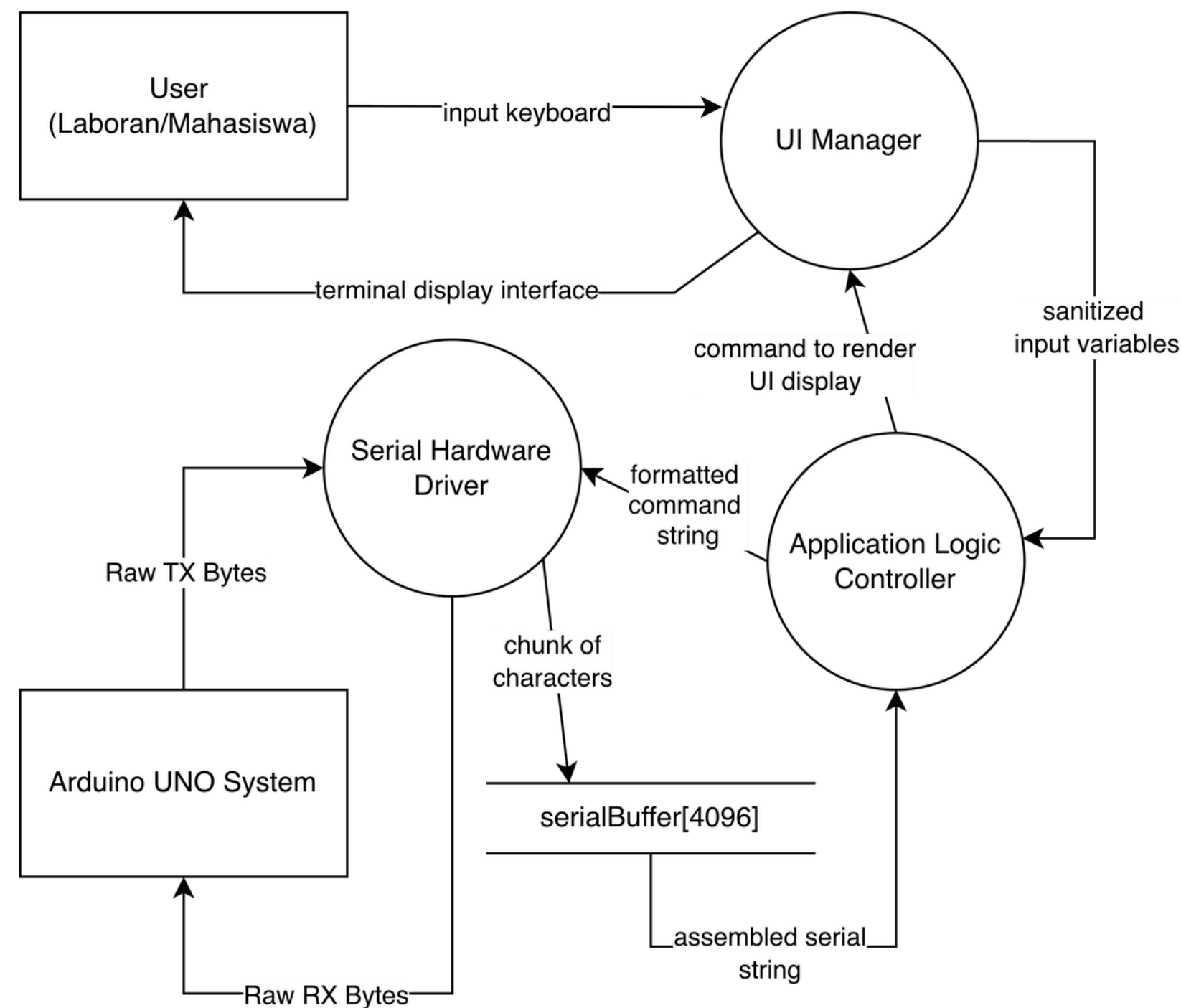
DFD Level 2



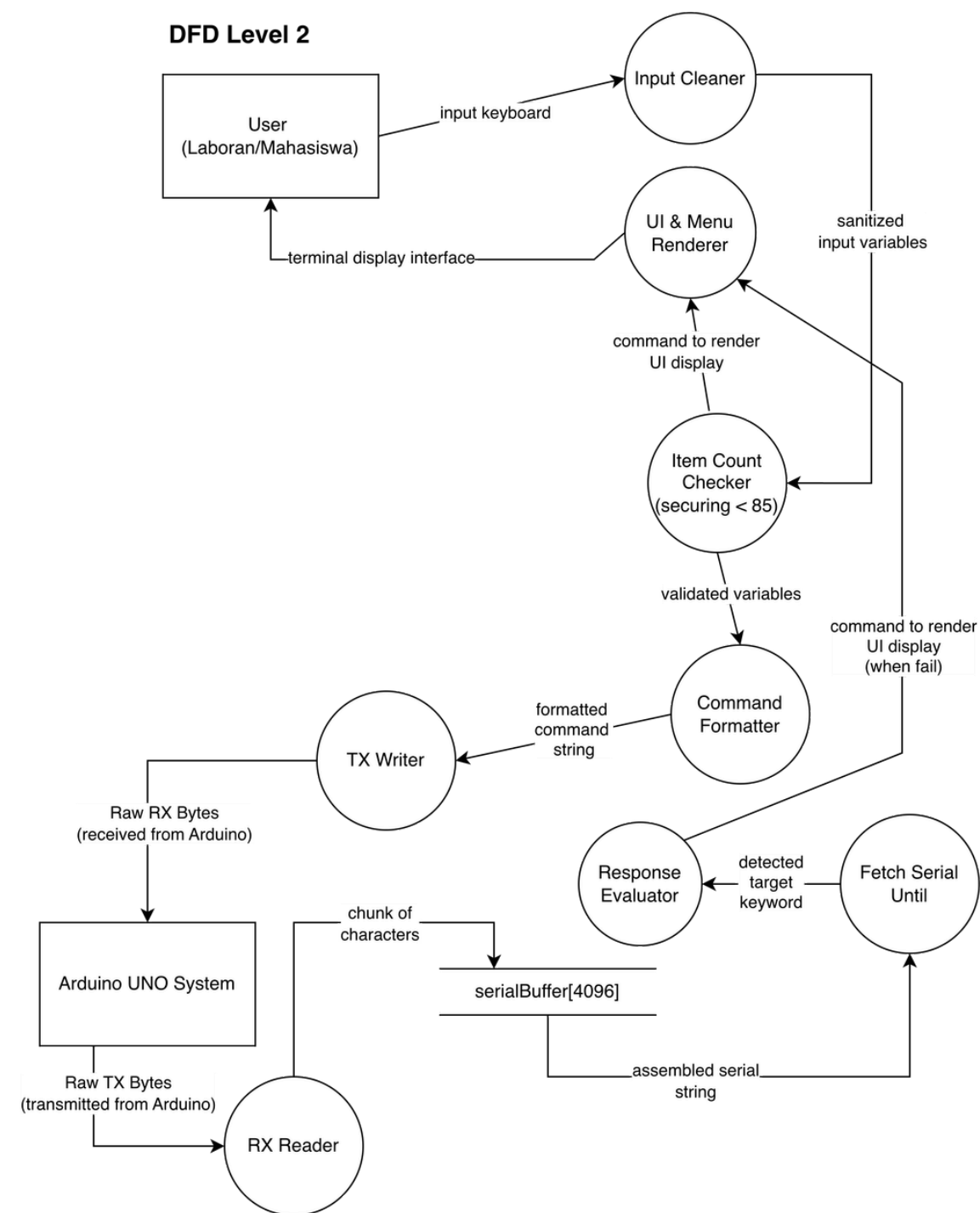
DFD Level 0 Client Interface System



DFD Level 1 Client Interface System

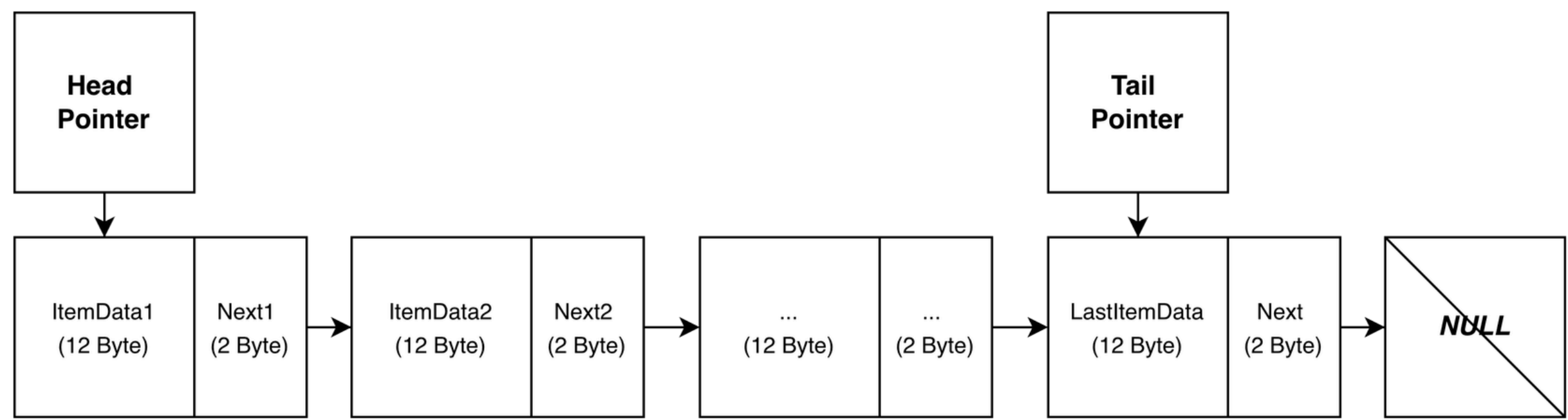


DFD Level 2 Client Interface System



Memori *Volatile*

Saat sistem berjalan, data inventaris tidak langsung diakses dari EEPROM, melainkan dimuat terlebih dahulu ke SRAM menggunakan struktur data ***Singly Linked List***. Pendekatan ini memungkinkan pengelolaan data secara dinamis dengan penggunaan memori yang tetap efisien.



Menggunakan Singly Linked List (FIFO) untuk penyimpanan data saat runtime.

Penambahan data $O(1)$ dengan bantuan tail pointer.

Setiap node terdiri dari 12 byte data inventaris dan 2 byte pointer, sehingga total 14 byte per item.

Kapasitas maksimum 85 item membutuhkan sekitar 1.190 byte (~58% SRAM Arduino Uno).

Sisa memori masih tersedia untuk buffer komunikasi serial dan variabel sementara program.

Memori *Non-Volatile*

Karena pointer pada Linked List tidak valid setelah mikrokontroler dimatikan, data di EEPROM disimpan sebagai Array of Structs yang tersusun secara berurutan. Untuk memaksimalkan kapasitas penyimpanan, digunakan struktur packed sehingga ukuran setiap data inventaris dapat dipadatkan menjadi 96 bit (12 byte) tanpa padding

Nama Variabel	Tipe Data	Ukuran Bit	Representasi
ID	Unsigned Integer	7 bit	Nilai 0-127
pName0 - pName7	Custom 6-Bit Char	48 bit	8 karakter
category	Unsigned Integer	2 bit	Nilai 0-3
location	Unsigned Integer	6 bit	Nilai 0-63
qtyTersedia	Unsigned Integer	6 bit	Kuantitas 0-63
qtyDipinjam	Unsigned Integer	6 bit	Kuantitas 0-63
qtyRusak	Unsigned Integer	6 bit	Kuantitas 0-63
pic	Custom 5-Bit Char	15 bit	3 karakter
Total Kapasitas		96 bit = 12 Byte	

Pemetaan kampus kompresi alfanumerik 6 bit:

Rentang Nilai (desimal)	Format Biner	Representasi Karakter
0 - 25	000000 - 011001	Huruf Uppercase 'A' hingga 'Z'
26 - 51	011010 - 110011	Huruf Lowercase 'a' hingga 'z'
52 - 61	110100 - 111101	Angka numerik '0' hingga '9'
62 dan 63	111110 & 111111	Karakter Spasi (padding) / NULL

Berikut merupakan struktur data yang digunakan:

```
struct __attribute__((packed)) ItemData {
    uint8_t id : 7;

    uint8_t pName0 : 6;
    uint8_t pName1 : 6;
    uint8_t pName2 : 6;
    uint8_t pName3 : 6;
    uint8_t pName4 : 6;
    uint8_t pName5 : 6;
    uint8_t pName6 : 6;
    uint8_t pName7 : 6;

    uint8_t category : 2;
    uint8_t location : 6;

    uint8_t qtyTersedia : 6;
    uint8_t qtyDipinjam : 6;
    uint8_t qtyRusak : 6;

    uint16_t pic : 15;};
```

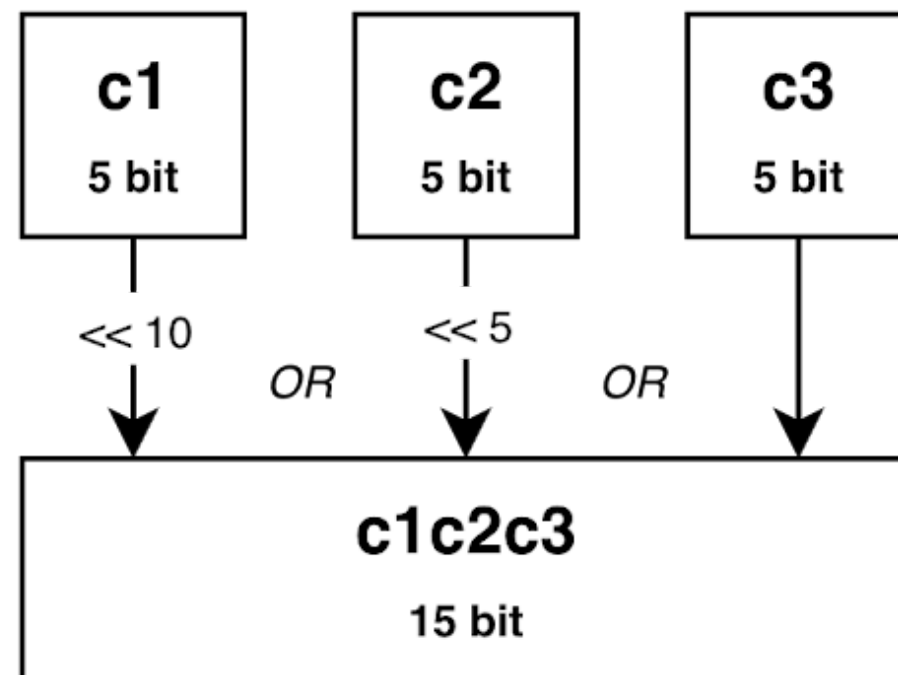
Pemetaan kampus kompresi 5 bit:

Rentang Nilai (desimal)	Format Biner	Representasi Karakter
0 - 25	00000 - 11001	Huruf Uppercase 'A' hingga 'Z'
26	11010	Karakter Spasi (padding) / NULL
27 - 31	11011 - 11111	Tidak Digunakan

Memori *Non-Volatile*

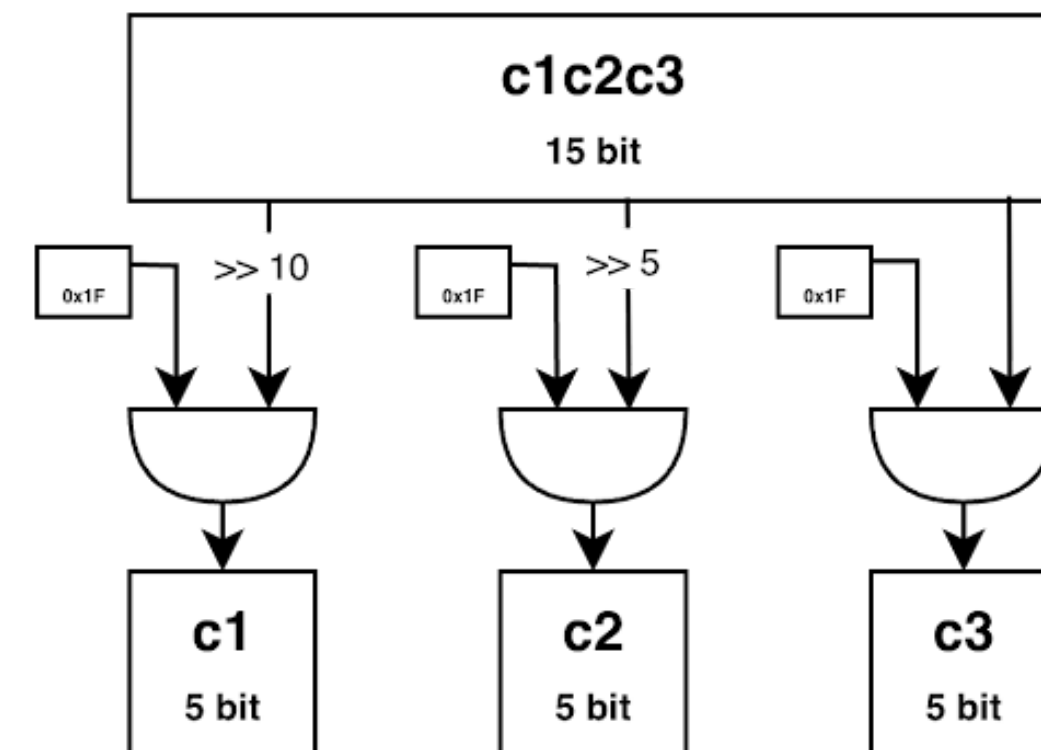
Proses Packing (compress ke EEPROM)

Untuk menghemat memori, karakter input tidak disimpan dalam format ASCII biasa, melainkan dikompresi menggunakan encoding 5-bit atau 6-bit sesuai kebutuhan. Pada atribut PIC, tiga karakter hasil encoding digabung menjadi satu data 15-bit menggunakan operasi bit shifting dan bitwise OR, sehingga penggunaan ruang penyimpanan menjadi lebih efisien.



Proses Unpacking (decompress untuk output UART):

Saat data inventaris ditampilkan, informasi yang tersimpan dalam format terkompresi terlebih dahulu didekode kembali menjadi string yang dapat dibaca pengguna. Proses ini dilakukan menggunakan teknik bitmasking dan bit shifting untuk mengekstrak data dari struktur 12-byte, kemudian menerjemahkannya kembali ke karakter ASCII melalui fungsi decode.



GET ALL (Tampilkan Seluruh Data)

Pilihan (1-7): 5

ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC
10	ESP32	Mikrokontroler	12	2	0	0	AMR
2	RASPI	Mikrokontroler	2	2	1	0	MAH
1	LIDAR	Aktuator	45	2	0	0	HTN
5	LIDAR2	Aktuator	45	1	1	0	HTN
12	ESP31	Aktuator	12	10	2	0	YTR
54	ESP33	Aktuator	24	5	0	0	AMR
21	LIDAR4	Aktuator	33	1	1	0	HTN
22	LIDAR5	Aktuator	33	2	0	0	HTH

SISTEM INVENTARISASI LABORATORIUM
1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar

Pilihan (1-7):

Search by ID (sukses)

ID	Nama	Kategori	Lokasi	Tersedia	Dipinjam	Rusak	PIC
2	RASPI	Mikrokontroler	2	2	1	0	MAH
1	LIDAR	Aktuator	45	2	0	0	HTN
5	LIDAR2	Aktuator	45	1	1	0	HTN
12	ESP31	Aktuator	12	10	2	0	YTR
54	ESP33	Aktuator	24	5	0	0	AMR
21	LIDAR4	Aktuator	33	1	1	0	HTN
22	LIDAR5	Aktuator	33	2	0	0	HTH

SISTEM INVENTARISASI LABORATORIUM
1. Tambah Data 2. Hapus Data 3. Cari Data (ID) 4. Update Stok & Status 5. Tampilkan Semua 6. Ringkasan Inventaris 7. Keluar

Pilihan (1-7): 3

===== CARI BERDASARKAN ID =====
Masukkan ID: 2

===== DATA DITEMUKAN =====
ID : 2
Nama : RASPI
Kategori : Mikrokontroler
Lokasi : 2
Tersedia : 2
Dipinjam : 1
Rusak : 0
PIC : MAH

Search by ID [ERROR]

```
+-----+
|  SISTEM INVENTARISASI LABORATORIUM  |
+-----+
|  1. Tambah Data                      |
|  2. Hapus Data                      |
|  3. Cari Data (ID)                  |
|  4. Update Stok & Status            |
|  5. Tampilkan Semua                |
|  6. Ringkasan Inventaris            |
|  7. Keluar                          |
+-----+
```

Pilihan (1-7): 3

===== CARI BERDASARKAN ID =====

Masukkan ID: 99

ID tidak ditemukan.

Keterbatasan Sistem

1

Keterbatasan Kapasitas Memori Non-Volatile

- EEPROM Arduino Uno hanya 1 KB, sehingga kapasitas inventaris dibatasi hingga 85 item.
- Sistem akan menolak penambahan data baru jika kapasitas penuh.
- Tidak cocok untuk inventaris skala besar tanpa memori eksternal tambahan.

2

Restriksi Format Karakter dan Panjang String

- Nama barang hanya mendukung huruf, angka, dan spasi.
- Nama PIC hanya mendukung huruf kapital dan spasi.
- Panjang nama barang dibatasi 8 karakter, sedangkan PIC 3 karakter.
- Input yang tidak sesuai format akan ditolak oleh client interface.

3

Ketergantungan pada Client Interface

- Setiap kategori stok (Tersedia, Dipinjam, Rusak) dibatasi hingga 63 unit.
- ID barang dibatasi pada rentang 0–127.
- Barang dengan jumlah melebihi 63 unit harus dipecah menjadi beberapa ID.

4

Batas Kuantitas Barang dan Rentang ID

- Operasi penghapusan data dapat menyebabkan delay karena perlu melakukan sinkronisasi dan pergeseran data.
- Waktu proses meningkat seiring bertambahnya jumlah item dalam Linked List.

5

Delay pada Operasi I/O

- Arduino bertindak sebagai database engine pasif.
- Validasi input sepenuhnya dilakukan oleh Client Interface.
- Tidak dirancang untuk penggunaan langsung melalui Serial Monitor.

Kesimpulan & Saran

Kesimpulan

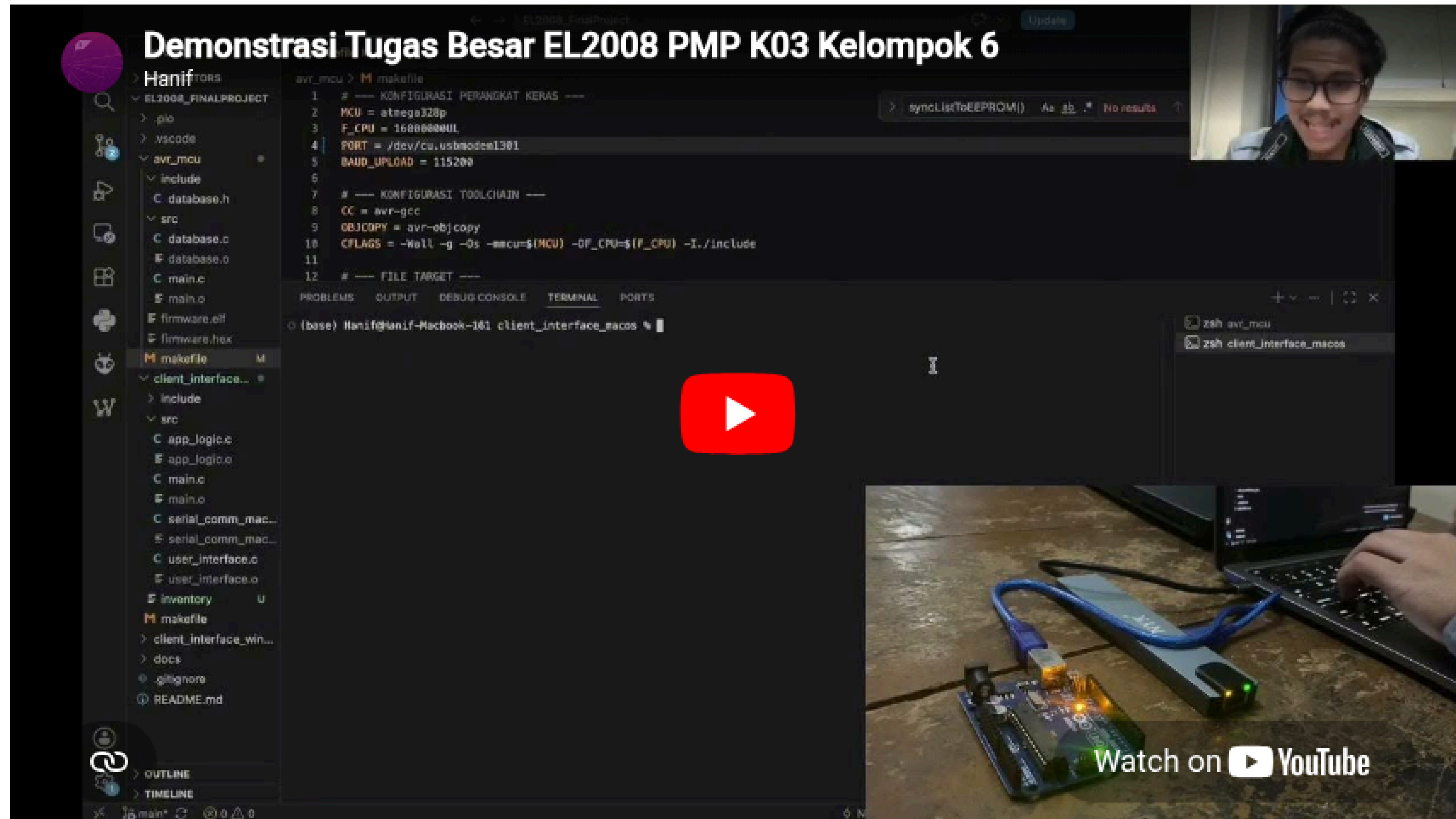
Sistem Inventarisasi Laboratorium berbasis Client-Server berhasil diimplementasikan pada Arduino Uno dengan memanfaatkan teknik optimasi memori tingkat rendah. Melalui kombinasi pengelolaan data di SRAM, penyimpanan permanen di EEPROM, dan kompresi data berbasis bit, sistem mampu bekerja secara stabil serta efisien pada keterbatasan sumber daya mikrokontroler.

- **Arsitektur Dual-Memory**
Menggunakan SRAM (Singly Linked List) untuk operasi data dinamis dan EEPROM untuk penyimpanan non-volatile.
- **Kompresi Bit-Level**
Struktur data berhasil dipadatkan menjadi hanya 12 byte per item melalui teknik bit-field dan encoding karakter.
- **Optimalisasi Kapasitas**
EEPROM 1 KB mampu menyimpan hingga 85 item inventaris.
- **Konsistensi dan Keamanan Data**
Client Interface melakukan validasi input dan pemantauan kapasitas memori secara real-time untuk mencegah data tidak valid maupun kelebihan kapasitas.

Saran

- ✓ **Pengembangan GUI** untuk meningkatkan kemudahan penggunaan dibandingkan antarmuka CLI saat ini.
- ✓ **Validasi ganda pada Arduino dan Client Interface** untuk meningkatkan keandalan serta mencegah data tidak valid.
- ✓ **Penambahan fitur keamanan dan autentikasi**, seperti RFID atau password admin, untuk membatasi akses operasi penting.
- ✓ **Integrasi QR Code Scanner** agar proses identifikasi dan pengelolaan barang dapat dilakukan secara otomatis.

Demonstrasi



<https://youtu.be/HQB-7MvPhCU?si=FYzQWlvf9uIOrDTA>

Pembagian Kerja

Pembagian Kerja Laporan

PEMBAGIAN TUGAS		
No	Komponen Tugas	PIC
A. LAPORAN		
1	Deskripsi Permasalahan	Nessa (13224078)
2	Dekomposisi & Solusi Permasalahan	Nessa (13224078)
3	Struktur Data & Logika	Hanif (13224087)
4	Flowchart	terdapat di list flowchart
5	DFD	Hanif (13224087)
6	Studi Kasus	Chrisjo and nagel
7	Analisis & Justifikasi	Chrisjo and nagel
8	Analisis Keterbatasan Sistem	Hanif (13224087)
9	Pembagian Tugas	bersama
B. Slides Presentasi		
1	Deskripsi & Dekomposisi Masalah	Nessa (13224078)
2	Deskripsi Sistem	Nessa (13224078)
3	Arsitektur Memori Arduino	Nessa (13224078)
4	Studi Kasus	Nessa (13224078)
5	Keterbatasan Sistem	Nessa (13224078)
6	Kesimpulan & Analisis Hasil	Nessa (13224078)
7	Demonstrasi	bersama
8	Pembagian Kerja & Kontribusi	

Pembagian Kerja Flowchart

Flowchart		
No	Nama Fungsi	PIC
app_logic.c		
1	runApp	Nagel (13224015)
2	addInventory	Nagel (13224015)
3	deleteInventory	Nagel (13224015)
4	searchByID	Nagel (13224015)
5	updateStockAndStatus	Chrisjo (13224108)
6	displayAllInventory	Chrisjo (13224108)
7	displaySummary	Chrisjo (13224108)
8	fetchSerialUntil	Nessa (13224078)
database.c		
1	initUART	Nessa (13224078)
2	printString	Chrisjo (13224108)
3	initDatabase	Nessa (13224078)
4	processSerialInput	Nagel (13224015)
5	encodeChar6Bit	Nessa (13224078)
6	decodeChar6Bit	Nessa (13224078)
7	encodeChar5Bit	Nessa (13224078)
8	decodeChar5Bit	Nessa (13224078)
9	packItemData	Nessa (13224078)
10	unpackItemName	Nessa (13224078)
database.c		
11	unpackItemPIC	Nessa (13224078)
12	insertItem	Chrisjo (13224108)
13	deleteItem	Chrisjo (13224108)
14	updateItem	Chrisjo (13224108)
15	syncListToEEPROM	Chrisjo (13224108)
16	printInventory	Chrisjo (13224108)
17	parseAndAddCommand	Chrisjo (13224108)
18	dataAvailableUART	Chrisjo (13224108)
19	receiveUART	Chrisjo (13224108)
20	transmitUART	Chrisjo (13224108)
serial_comm		
1	initSerialPort	Hanif (13224087)
2	sendSerialData	Hanif (13224087)
3	receiveSerialData	Hanif (13224087)
4	closeSerialPort	Hanif (13224087)
user_interface (DONE)		
1	printMainMenu	Nessa (13224078)
30	getUserChoice	Nessa (13224078)
31	promptString	Nessa (13224078)
32	promptInt	Nessa (13224078)
33	printTableHeader	Nessa (13224078)
34	printTableFooter	Nessa (13224078)

Pembagian Kerja

Pembagian Kerja Program C

Pembuatan Program C		
No	Nama Fungsi	PIC
app_logic.c		
1	runApp	Nagel (13224015)
2	addInventory	Nagel (13224015)
3	deleteInventory	Nagel (13224015)
4	searchByID	Nagel (13224015)
5	updateStockAndStatus	Chrisjo (13224108)
6	displayAllInventory	Chrisjo (13224108)
7	displaySummary	Chrisjo (13224108)
8	fetchSerialUntil	Chrisjo (13224108)
database.c		
1	initUART	Hanif (13224087)
2	printString	Chrisjo (13224108)
3	initDatabase	Hanif (13224087)
4	processSerialInput	Chrisjo (13224108)
5	encodeChar6Bit	Hanif (13224087)
6	decodeChar6Bit	Hanif (13224087)
7	encodeChar5Bit	Hanif (13224087)
8	decodeChar5Bit	Hanif (13224087)
9	packItemData	Hanif (13224087)
10	unpackItemName	Hanif (13224087)

database.c		
10	unpackItemName	Hanif (13224087)
11	unpackItemPIC	Hanif (13224087)
12	insertItem	Chrisjo (13224108)
13	deleteItem	Chrisjo (13224108)
14	updateItem	Chrisjo (13224108)
15	syncListToEEPROM	Hanif (13224087)
16	printInventory	Chrisjo (13224108)
17	parseAndAddCommand	Chrisjo (13224108)
18	dataAvailableUART	Hanif (13224087)
19	receiveUART	Hanif (13224087)
20	transmitUART	Hanif (13224087)
serial_comm		
1	initSerialPort	Hanif (13224087)
2	sendSerialData	Hanif (13224087)
3	receiveSerialData	Hanif (13224087)
4	closeSerialPort	Hanif (13224087)
user_interface		
1	printMainMenu	Nessa (13224078)
2	getUserChoice	Nessa (13224078)
3	promptString	Nessa (13224078)
4	promptInt	Nessa (13224078)
5	printTableHeader	Nessa (13224078)
6	printTableFooter	Nessa (13224078)

REVISI URGENT			
Problem	Files	PIC	Status
Di deleteInventory dan updateStockStatus, belum ada validasi ID jika tidak ada di database	app_logic.c	Nagel	☒
Validasi input string yang digunakan (sesuaikan dengan ketentuan karakter nama item dan ketentuan karakter nama PIC.	app_logic.c	Nagel	☒
Di interface, saat input, belum dikasih keterangan untuk arti nomor kategori itu apa (perlu dikasih rincian di tampilan informasi input)	app_logic.c	Nagel	☒
Di interface, saat output, belum dikasih arti aktual untuk jenis kategori (misal: 0 = microcontroller, dst...)	app_logic.c	Nagel	☒
- ketika sudah max 85. Lalu ditambahkan 1 item baru (jadinya 86 item). Lalu di delete 1 item sebelumnya, secara logika seharusnya item ke-86 mundur ke-85 (sehingga tersimpan di eeprom). Namun ketika disave dan dicabut, lalu diperiksa kembali, item ke-86 tadi tidak tersimpan. Hal yang sama terjadi ketika 1 item dihapus terlebih dahulu, lalu ditambahkan item terbaru. Item terbaru ini tidak tersimpan di eeprom	database.c	Hanif	☒
masih terdapat fungsi yang belum bertipe void, perlu diganti	database.c	Nagel	☒
di updateStockAndStatus, belum ada validasi input kuantitas harus di rentang 0-63	app_logic.c	Nagel	☒

Thank You!